

5

Enabling conversational memory

This chapter covers

- Maintaining conversational state
- In-memory chat history
- Retaining long-term memory

Have you seen the movie *50 First Dates*? In it, Drew Barrymore’s character, Lucy, lives with a brain injury sustained in a car accident that gives her a type of short-term memory loss. In the movie, her memory resets to the time just before her accident every day when she wakes up. She retains no memory of anything that has happened since the accident after she goes to sleep at night.

In the same movie, there’s another character called “10-Second Tom.” Tom has a similar injury (from a hunting accident) in which his memory is reset every 10 seconds. Whereas Lucy’s slightly longer memory drives the plot of the movie, Tom’s condition is more comical, with him reintroducing himself by saying, “Hi, I’m Tom,” every 10 seconds.

Even though LLMs seem to be incredibly smart and able to answer almost any question, they have memory problems not unlike those experienced by Lucy and 10-Second Tom. In fact, Lucy’s and Tom’s memories are vastly superior to an LLM’s

memory: the LLM doesn't retain any memory past the completion of a prompt. In short, LLMs are stateless services.

This extreme case of short-term memory loss makes it challenging to conduct a conversation with an LLM. If you asked, "Why is the sky blue?" and follow that question with "Is it ever orange?", the LLM would have no idea what "it" refers to in the second question, because it has forgotten that you ever asked a question about the sky before.

Fortunately, Spring AI has a solution to the LLM's extremely short-term memory problems. And, as it turns out, that solution involves using advisors much like how Spring AI implements RAG using `QuestionAnswerAdvisor`. Let's see how it works.

5.1 *Making memories in AI*

If you were having trouble remembering something, what might you do to help you remember? Tie a string around your finger? Employ a mnemonic device? Or simply write it down in a note to your future self to read?

You can't tie a string around an LLM's finger (chiefly because they don't have fingers), and mnemonic devices are typically based on human whimsy, which is something LLMs are decidedly lacking in. But jotting down something in a note can be useful for both humans and LLMs. So that's the trick Spring AI employs to help an LLM remember a conversation.

When the user asks a question to the LLM in the course of a conversation, the user message carrying the question is stored in some form of memory (defined in Spring AI by the `ChatMemory` interface). Then, when the answer comes back, the LLM's response message (called an "assistant" message) is also stored away for future reference.

The next time the user asks a question, the same thing happens again, but first, the user and assistant messages from past exchanges in the same conversation are pulled from memory and written into the prompt as context, similar to how documents are pulled from a vector store and written in the prompt context in RAG. Figure 5.1 illustrates this exchange and how user and assistant messages are stored in memory and the chat history, which are retrieved and used as context in prompts to the LLM.

As a result, the conversation is stored as a dialog between the user and the assistant and played back to the LLM on every request. The LLM is reminded of everything that has been said before, which enables it to continue the conversation and stay on topic.

Spring AI implements management of conversational memory as advisors. More specifically, Spring AI provides three different advisors, each with its own strategy for managing the history of the chat:

- `MessageChatMemoryAdvisor`
- `PromptChatMemoryAdvisor`
- `VectorStoreChatMemoryAdvisor`

Both `MessageChatMemoryAdvisor` and `PromptChatMemoryAdvisor` store the chat history via some implementation of `ChatMemory`, but they differ in how the prompt is populated with chat history. `MessageChatMemoryAdvisor` adds the chat history to the prompt as distinct messages for the user and assistant roles. But not all models support

role-based messages. For those models, `PromptChatMemoryAdvisor` will inject the chat history into the prompt as one big string injected into a system message template.

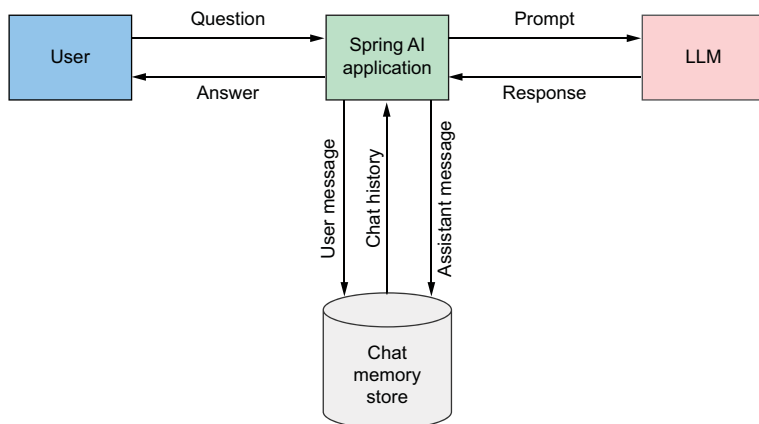


Figure 5.1 Chat memory keeps a record of user and LLM interactions as a reminder for future prompts.

`VectorStoreChatMemoryAdvisor` takes a completely different approach to storing chat history by storing it in a vector store. By doing this, it can query the vector store, the same way it would query for document chunks in RAG, finding the pieces of chat history that are most similar to the question being asked. Then it places the similar snippets of chat history into the prompt as a string injected into a system message template, the same as `PromptChatMemoryAdvisor`.

To see how each of these advisors works, let's start with a simple example that shows how you can use Spring AI's chat memory advisors to add conversational memory to the Board Game Buddy application.

5.2 Adding conversational memory

When someone is asking questions about the rules of a game, it wouldn't be uncommon for them to ask one question and then ask one or more follow-up questions. For example, when asking about Burger Battle, you might ask a question about one of the battle cards, followed by a bit of clarification on how it can be used. For example, consider these two questions:

- "What is the Burger Force Field card?"
- "Does it protect against Burgerpocalypse?"

Even though this exchange is brief, it still demonstrates the importance of maintaining a memory of the conversation. The first question is very specific about the subject (the Burger Force Field card), while the second question refers to the subject with a

pronoun (“it”). Without the context established in the first question, the second question doesn’t make much sense.

Spring AI’s `MessageChatMemoryAdvisor` and `PromptChatMemoryAdvisor` are both quite capable of handling this kind of discussion by storing every question asked by the user, along with every answer provided by the LLM, and then replaying the script to the LLM along with the next question being asked.

Let’s see how these advisors are used by adding them as a default advisor when creating the `ChatClient`.

5.2.1 *Enabling an in-memory chat advisor*

If you’re looking forward to writing a lot of code to enable chat memory in the application, you are going to be disappointed. On the other hand, if you like it when a small change yields big results, you’re going to really enjoy what happens next.

As it turns out, adding chat memory is a simple matter of adding one more default advisor when creating the `ChatClient`. The following listing shows what’s needed to make it work.

Listing 5.1 Adding message-oriented chat memory as a default advisor

```
@Bean
ChatClient chatClient(
    ChatClient.Builder chatClientBuilder,
    VectorStore vectorStore,
    ChatMemory chatMemory) {
    return chatClientBuilder
        .defaultAdvisors(
            MessageChatMemoryAdvisor.builder(chatMemory).build(),
            QuestionAnswerAdvisor.builder(vectorStore)
                .searchRequest(SearchRequest.builder().build()).build()
        )
        .build();
}
```

Adds the default
MessageChatMemory
Advisor

Although there’s a lot that goes on behind the scenes regarding chat memory management, the addition of a `MessageChatMemoryAdvisor` to `defaultAdvisors()` is all that’s needed to enable simple chat memory. The `MessageChatMemoryAdvisor` is created, given the `ChatMemory` passed into the `chatClient()` method (which is auto-configured as an instance of `MessageWindowChatMemory`), and adds the advisor to the list of default advisors alongside the `QuestionAnswerAdvisor` you added in the previous chapter. By default, `MessageWindowChatMemory` uses an in-memory implementation to store conversation history.

Alternatively, you could choose to specify a `PromptChatMemoryAdvisor` in a very similar manner:

```
@Bean
ChatClient chatClient(
    ChatClient.Builder chatClientBuilder,
    VectorStore vectorStore,
    ChatMemory chatMemory) {
```

```

return chatClientBuilder
    .defaultAdvisors(
        PromptChatMemoryAdvisor.builder(chatMemory).build(),
        QuestionAnswerAdvisor.builder(vectorStore)
            .searchRequest(SearchRequest.builder().build()).build())
        .build();
}

```

In either case, the main thing to take note of is when the advisor is created, it is given an instance of `MessageWindowChatMemory`. This implementation of Spring AI's ChatMemory interface relies on a `ChatMemoryRepository` for stowing chat messages. By default, the implementation of `ChatMemoryRepository` stores the conversation history in memory in a simple `Map`. Because the chat history is maintained in a `Map` and not written to a database or otherwise stored outside of the application, it will not survive an application restart. But for many use cases, including just-getting-started situations, it is a fine choice.

Now that you have given the Board Game Buddy application the ability to remember the conversation, let's try it out. Fire up the application and try asking about the Burger Force Field card from Burger Battle (assuming that you have loaded the Burger Battle rules into the vector store):

```

$ http :8080/ask gameTitle="Burger Battle" \
      question="What is the Burger Force Field card?" -b
{
  "answer": "The Burger Force Field card is a Battle Card that protects
    your Burger from all other Battle Cards.",
  "gameTitle": "Burger Battle"
}

```

Great! Now let's ask a follow-up question that requires some past context from the chat history:

```

$ http :8080/ask gameTitle="Burger Battle" \
      question="Does it protect against Burgerpocalypse?" -b
{
  "answer": "The Burger Force Field card does not protect against
    Burgerpocalypse.",
  "gameTitle": "Burger Battle"
}

```

Even though the follow-up question doesn't mention the Burger Force Field card, it's clear that the answer refers to that card. Chat memory works, and it only took one additional advisor to make it happen!

Let's take a peek under the hood to see how the chat memory was communicated to the LLM in each request.

5.2.2 Inspecting the prompt for chat memory

Up to this point, you've used user messages to carry questions and text from the user and system messages to carry instructions that come from the application itself. When

using one of the chat memory advisors, you're also using assistant messages that represent the response from the LLM. Chat history is a collection of user and assistant messages where the user message is a question that was asked and the assistant message is the answer.

With both `MessageChatMemoryAdvisor` and `PromptChatMemoryAdvisor`, the user and assistant messages are stored in chat memory the same way. The key difference between the two advisors is in how those messages from chat memory are sent to the LLM in a prompt.

EXAMINING THE MESSAGECHATMEMORYADVISOR PROMPT

In the case of `MessageChatMemoryAdvisor`, the user and assistant messages are sent as-is in the prompt. For example, if the conversation has just started, there are no messages in chat memory. So, if the user asks about the Burger Force Field card, the prompt JSON sent looks like this (for OpenAI's GPT-4o):

```
{
  "messages" : [ {
    "content" : [ {
      "type" : "text",
      "text" : "What is the Burger Force Field?\nContext information
              is below.\n-----\n
              HOW TO PLAY\n\nBURGER BATTLE\n\n2-6 players ... "
    } ],
    "role" : "user"
  } ],
  "model" : "gpt-4o",
  "stream" : false,
  "temperature" : 0.7
}
```

Look closely at the "messages" property, and you can see that there is only one message. It is a user message that carries both the current question and the RAG context (truncated here to save space).

After the first question has been asked and answered, the user message with the question and the assistant message that contains the answer are stored in chat memory. Then, when the follow-up question is asked, the prompt JSON looks like this:

```
{
  "messages" : [ {
    "content" : [ {
      "type" : "text",
      "text" : "What is the Burger Force Field?"
    } ],
    "role" : "user"
  }, {
    "content" : [ {
      "type" : "text",
      "text" : "{\n  \"answer\": \"The Burger Force Field is a Battle
              Card that protects your Burger from all other Battle Cards.
              \",\n  \"gameTitle\": \"Burger Battle\"\n}"
    } ],
    "role" : "assistant"
  } ],
  "model" : "gpt-4o",
  "stream" : false,
  "temperature" : 0.7
}
```

```

    } ],
    "role" : "assistant"
  }, {
    "content" : [ {
      "type" : "text",
      "text" : "Does it protect against Burgerpocalypse?\nContext information
               is below.\n-----\n
               HOW TO PLAY\n\nBURGER BATTLE\n\n2-6 plavers ... "
    } ],
    "role" : "user"
  } ],
  "model" : "gpt-4o",
  "stream" : false,
  "temperature" : 0.7
}

```

The "messages" property has now grown to three entries: A user message with the original question, an assistant message with the answer, and another user message with the new question and RAG context. For every question asked during the conversation, the "messages" property would grow by two more entries: an assistant message with the answer for the most recently asked question and a user message with the newly posed question.

EXAMINING THE PROMPTCHATMEMORYADVISOR PROMPT

As for PromptChatMemoryAdvisor, rather than add user and assistant messages to the prompt for each turn in the conversation, the entire conversation is captured as a String and injected into a system prompt as context.

To see how this works, let's say that the conversation has just begun and chat memory is empty. After asking the first question, the prompt looks like this:

```

{
  "messages" : [ {
    "content" : [ {
      "type" : "text",
      "text" : "\nUse the conversation memory from the MEMORY section to
               provide accurate answers.\n\n-----\nMEMORY:\n
               -----\n\n"
    } ],
    "role" : "system"
  }, {
    "content" : [ {
      "type" : "text",
      "text" : "What is the Burger Force Field?\nContext information
               is below.\n-----\nHOW TO PLAY\n\nBURGER ..."
    } ],
    "role" : "user"
  } ],
  "model" : "gpt-4o",
  "stream" : false,
  "temperature" : 0.7
}

```

Here, you see that there are two messages: A system message that tells the LLM to use conversation memory and a user message that asks the question (along with RAG context). Because the conversation has just begun, there is no conversation memory to provide in the system message.

When the follow-up question is asked, there will now be some conversation memory. The prompt will look like this:

```
{
  "messages" : [ {
    "content" : [ {
      "type" : "text",
      "text" : "\nUse the conversation memory from the MEMORY section to
        provide accurate answers.\n\n-----\nMEMORY:\n
        USER:What is the Burger Force Field?\n
        ASSISTANT:{\n  \"answer\": \"The Burger Force Field is a Battle
        Card that protects your Burger from all other Battle Cards.\",\n
        \"gameTitle\": \"Burger Battle\"\n}\n-----\n\n"
    } ],
    "role" : "system"
  }, {
    "content" : [ {
      "type" : "text",
      "text" : "Does it protect against Burgerpocalypse?\nContext information
        is below.\n-----\nHOW TO PLAY\n\nBURGER BATTLE..."
    } ],
    "role" : "user"
  } ],
  "model" : "gpt-4o",
  "stream" : false,
  "temperature" : 0.7
}
```

There are still only two messages, as before. But the system message now includes the text from the original user question and the answer from the assistant. The LLM will use this to answer the new question in the user message.

As was the case with `MessageChatMemoryAdvisor`, the conversation memory will grow as the conversation proceeds. But instead of adding multiple messages to the prompt, the text of the system message will grow to include the dialogue of the conversation that has passed.

When only asking a handful of questions, there's not much concern about the conversation history getting too big. But as the conversation carries on, the chat memory grows in length. And when sending that chat memory as context in the prompt, it counts against the usage tokens. That means that as the conversation goes on, the token cost goes up. And, of course, there's a risk that the prompt could eventually exceed the context window limit if the conversation goes on for a very long time. To avoid the problem of an ever-growing chat history, let's see how to gain some control over how much of the history is sent in the prompt.

5.2.3 Configuring chat memory size

Without you doing anything at all, Spring AI already has you covered when it comes to limiting chat history sent in a prompt. By default, only the most recent 20 messages are sent from the chat memory. After 50 exchanges between the user and the assistant, old messages will drop off when new messages are added, ensuring that the chat memory sent in the prompt will not exceed 20 messages.

You may, however, wish to adjust that limit, either setting it higher for greater conversational context or setting it lower to avoid greater token usage. Regardless of your reason for adjusting it, setting the limit involves overriding the autoconfigured `ChatMemory` bean to set custom configuration for the `MessageWindowChatMemory` that is created.

For example, suppose that you decide to limit the number of messages in chat memory to 50. You can set the chat history limit when creating the `ChatMemory` bean by explicitly defining a `@Bean` method that creates a `ChatMemory`. In that method, create a `MessageWindowChatMemory` instance using its builder and calling `maxMessages()`. Here's what that `@Bean` method would look like:

```
@Bean
ChatMemory chatMemory(ChatMemoryRepository chatMemoryRepository) {
    return MessageWindowChatMemory.builder()
        .chatMemoryRepository(chatMemoryRepository)
        .maxMessages(50)
        .build();
}
```

Proving that the chat history limit works can be somewhat tedious. You'd need to carry on a lengthy conversation and inspect the prompt to confirm that it works. But if you were to ask 25 questions and get 25 answers, then the 26th question would not be able to use context from the first question, as it would be dropped. Older conversation entries will be forgotten as new entries are added. Even so, it's reasonable to think that more recent conversation entries include enough information to infer what's needed.

Next, let's have a look at how to set a conversation ID to manage multiple distinct conversations.

5.3 Specifying the conversation ID

Each conversation has an ID associated with it. If you don't specify otherwise, the conversation ID will be default. That's fine if your application only has one user. But chances are you have many users, and you'll want to keep their conversations distinct from each other. To do that, you'll need to assign a unique conversation ID.

The conversation ID can be any `String` value that suits you for keeping conversations separate. It could be a user's username, the session ID, or perhaps the value of a request header, leaving it up to the client to specify. The following listing shows a modification to the controller's `ask()` method that extracts the conversation ID from a custom request header whose name is `X_AI_CONVERSATION_ID`.

Listing 5.2 Pulling the conversation ID from a request header

```

@PostMapping(path = "/ask", produces = "application/json")
public Answer ask(
    @RequestHeader(name="X_AI_CONVERSATION_ID",
        defaultValue = "default") String conversationId,
    @RequestBody @Valid Question question) {
    return boardGameService.askQuestion(question, conversationId);
}

```

**Injects
conversation
ID header** ←
 ←
**Passes conversation
ID to service**

Note that `X_AI_CONVERSATION_ID` is a custom header defined for the purposes of this application. It is not something specific to Spring AI, and you are welcome to name it anything you want.

After receiving the conversation ID from the request header, it passes it along with the question to `SpringAiBoardGameService`'s `askQuestion()` method. That means that `SpringAiBoardGameService` needs to change to accept the conversation ID and use provide it to keep track of the conversation. First, the `BoardGameService` will need to change to accept the conversation ID as a parameter:

```

package com.example.boardgamebuddy;

public interface BoardGameService {
    Answer askQuestion(Question question, String conversationId);
}

```

Then you'll need to make the appropriate changes to `SpringAiBoardGameService`. The following listing shows what is needed to add conversational capabilities.

Listing 5.3 Providing the conversation ID to the advisor

```

import static org.springframework.ai.chat.memory
    .ChatMemory.CONVERSATION_ID;

// ...

@Override
public Answer askQuestion(Question question, String conversationId) {
    var gameNameMatch = String.format(
        "gameTitle == '%s'",
        normalizeGameTitle(question.gameTitle()));

    return chatClient.prompt()
        .system(systemSpec -> systemSpec
            .text(promptTemplate)
            .param("gameTitle", question.gameTitle()))
        .user(question.question())
        .advisors(advisorSpec -> advisorSpec
            .param(FILTER_EXPRESSION, gameNameMatch)
            .param(CONVERSATION_ID, conversationId))
        .call()
        .entity(Answer.class);
}

```

**Sets
conversation ID** ←

In the previous version of `askQuestion()`, the `advisors()` method was called to specify that `QuestionAnswerAdvisor` is to be used for performing RAG on the request. That's still here in this version of `askQuestion()`, but now another `advisors()` method is also called that accepts a `Consumer<AdvisorSpec>`. In this case, the provided `AdvisorSpec` is used to set the conversation ID as a parameter to the advisors. The `CONVERSATION_ID` constant provides the key that is shared with the internal chat memory advisor implementations, so that you don't have to remember to make sure that the key matches what they expect.

As for the conversation ID value, the `ask()` method accepts it as a parameter annotated with `@RequestHeader` to extract it from the request's `X_AI_CONVERSATION_ID` header. If the header doesn't exist, it will set the conversation ID to `default`. The `ask()` method then passes the conversation ID as a parameter when it calls `askValue()` on the service.

Now, restart the application and try it out. To ensure that the conversation ID is set, specify the `X_AI_CONVERSATION_ID` header in the request. When using `HTTPie`, request headers are specified by giving their name and value, separated by a colon. Here's how you can submit that initial question within the context of a conversation whose ID is `conversation_1`:

```
$ http :8080/ask gameTitle="Burger Battle" \
      question="What is the Burger Force Field card?" \
      X_AI_CONVERSATION_ID:conversation_1 -b
{
  "answer": "The Burger Force Field card is a Battle Card in the Burger
            card game that allows a player's Burger to be protected from
            all other Battle Cards.",
  "gameTitle": "Burger Battle"
}
```

So far, so good. Now, ask a follow-up question:

```
$ http :8080/ask gameTitle="Burger Battle" \
      question="Does it protect against Burgerpocalypse?" \
      X_AI_CONVERSATION_ID:conversation_1 -b
{
  "answer": "The Burger Force Field card does not protect against
            Burgerpocalypse as it specifically states that all players'
            ingredients are destroyed, regardless of protection.",
  "gameTitle": "Burger Battle"
}
```

As before, it's clear that the LLM was able to infer that “it” means “Burger Force Field card” from the conversation history. As such, it was able to answer the question without having the Burger Force Field card be explicitly mentioned.

Now, let's trip it up by switching conversations to `conversation_2` but leaving the subject unclear as “it”:

```
$ http :8080/ask gameTitle="Burger Battle" \
      question="How do you remove it?" \
      X_AI_CONVERSATION_ID:conversation_2 -b
```

```
{  
  "answer": "To remove ingredients from your Burger, you can play Battle  
            Cards such as Burger Bomb or Picky Eater, which send  
            ingredients to the Graveyard.",  
  "gameTitle": "Burger Battle"  
}
```

While it did provide an answer, that answer was just as out of context as the question itself, talking about how ingredients can be removed from a burger. Since this request involved a brand-new conversation, there was no history to inform it that you're asking about the Burger Force Field card. It did its best but had to infer (incorrectly) what the question was referring to.

As you've seen, adding chat memory to a Spring AI application with either `MessageChatMemoryAdvisor` or `PromptChatMemoryAdvisor` is incredibly easy. They are a great start for enabling conversational history in a Spring AI application. But are they ready for production use?

The biggest problem with how you've used them so far is that they rely on `MessageWindowChatMemory`, whose default `ChatMemoryRepository` is an in-memory implementation and won't survive application restarts or shared across multiple application instances. Because the chat history is written to memory, it will continue to eat up application memory over time until the application is restarted.

As such, using either of those chat memory advisors with `MessageWindowChatMemory`'s default behavior is not recommended for use in a production setting. Instead, let's see how to add persistent chat memory to a Spring AI application that can survive application restarts, be shared across application instances, and not endlessly consume memory.

5.4 *Enabling persistent chat memory*

Adding a chat memory advisor to the Board Game Buddy application didn't give the LLM itself memory. But with the application's help, the LLM is now capable of engaging in conversation, recalling past exchanges, and using them to inform its next response. It has virtually broken free from 10-Second Tom's forgetfulness.

Although it's no longer like 10-Second Tom, it is still somewhat like Lucy from *50 First Dates*. Lucy was able to maintain memory throughout the day, but once she went to sleep at night, all memory of that day was lost, and she'd start over the next day with no recollection of the previous day. Similarly, when using `MessageWindowChatMemory` as you have so far, the application loses its memory when it stops and restarts with a blank slate.

Spring AI offers a couple of options to prevent chat memory from being like the memory of forgetful Lucy:

- Using one of Spring AI's persistent implementations of `ChatMemoryRepository`
- `VectorStoreChatMemoryAdvisor`

Let's start by taking a look at how to use Spring AI's persistent `ChatMemoryRepository` implementations.

5.4.1 Persisting chat memory to a database

As mentioned already, the default implementation of `ChatMemoryRepository` used by `MessageWindowChatMemory` is one that stores chat entries in memory. If you could plug in a different implementation of `ChatMemoryRepository` that persists chat entries to a more durable data store, the conversation history could endure past application restarts. While you could write your own, Spring AI comes with three such implementations for you to choose from:

- `CassandraChatMemoryRepository`
- `JdbcChatMemoryRepository`
- `Neo4jChatMemoryRepository`

Furthermore, thanks to autoconfiguration, it's extremely easy to add them to your project. We'll start by looking at how to add Cassandra-based chat memory to your Spring AI application.

CASSANDRA

Apache Cassandra is a popular high-volume NoSQL database; it is highly scalable and designed to handle data across multiple nodes. Its data structure is a hybrid between tabular data and key-value stores. Cassandra can be queried with `Cassandra Query Language (CQL)`, which is quite similar to SQL used to query traditional relational databases.

Spring AI's `CassandraChatMemoryRepository` is built to store chat history in a Cassandra cluster. Even if you don't know much about Cassandra, you can still use `CassandraChatMemoryRepository`, thanks to autoconfiguration that hides the nitty-gritty of working with Cassandra.

The first step to using Cassandra for chat memory is to add the following starter dependency to your project's build:

```
implementation 'org.springframework.ai:spring-ai-starter-model-chat-memory-repository-cassandra'
```

You can also add this dependency by choosing the `Cassandra Chat Memory Repository` option from the `Spring Initializr`.

Next, you'll need to change the `ChatMemory` bean you defined earlier to build the `MessageWindowChatMemory` instance with a reference to an injected `ChatMemoryRepository` bean:

```
@Bean
ChatMemory chatMemory(ChatMemoryRepository chatMemoryRepository) {
    return MessageWindowChatMemory.builder()
        .chatMemoryRepository(chatMemoryRepository)
        .build();
}
```

You won't need to define the `ChatMemoryRepository` bean, however. Spring AI autoconfiguration will create one for you, and it will be available for injection into the `chatMemory()` bean method when the application context is being created.

That's all that's required to use Cassandra for persistent chat memory. But you'll need to arrange for a Cassandra cluster to be available. For the purposes of Board

Game Buddy, we're already running our vector store from Docker Compose, so it'll be easy to add Cassandra to the mix. Just add the following service entry in the Docker Compose file:

```
services:
  cassandra:
    image: 'cassandra:latest'
    environment:
      - 'CASSANDRA_DC=dc1'
      - 'CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch'
    ports:
      - '9042:9042'
```

When you start up the application, chat history will be persisted to tables in the Cassandra cluster. By default, chat history is written to a table named `ai_chat_memory` in a column named `messages`. The table will be in the Cassandra namespace `springframework`. You can customize these details by setting configuration properties like this:

```
spring.ai.chat.memory.repository.cassandra.keyspace=boardgamebuddy
spring.ai.chat.memory.repository.cassandra.messages-column=chat_messages
spring.ai.chat.memory.repository.cassandra.table=chat_memory
```

In this example configuration, the table will be named `chat_memory`, the message column will be named `chat_messages`, and the namespace will be `boardgamebuddy`. This schema will be created for you automatically, so there's no need for you to worry about creating it yourself. But in a production setting, you may wish to manage the schema yourself. If so, you can disable schema initialization like this:

```
spring.ai.chat.memory.repository.cassandra.initialize-schema=false
```

While Cassandra is a solid choice for persistent chat memory, you might also consider storing chat history in the Neo4j graph database. Let's see how to do that.

Neo4j

Neo4j is a popular and powerful graph database. Data is stored in Neo4j in terms of entity nodes and the relationships between those entities. As you'll soon see, entities can be visualized as circles with lines connecting them that represent their relationships.

Using Neo4j for persistent chat history is much the same as using Cassandra, with only a few small changes. The main change involves using the Neo4j chat memory starter dependency instead of the Cassandra chat memory starter dependency:

```
implementation 'org.springframework.ai:spring-ai-starter-model-chat-memory-
  repository-neo4j'
```

You can also add this starter to your project by selecting the Neo4j Chat Memory Repository dependency from the Initializr.

The `ChatMemory` bean you defined for Cassandra will work equally well for Neo4j without any changes. However, the `ChatMemoryRepository` injected into the `chatMemory()` bean method will be a `Neo4jChatMemoryRepository`.

If you're relying on Docker Compose to start the Neo4j database, you'll need to edit `compose.yaml` to add the following Neo4j service:

```
services:
  neo4j:
    image: 'neo4j:latest'
    environment:
      - 'NEO4J_AUTH=neo4j/notverysecret'
    ports:
      - '7687:7687'
      - '7474:7474'
```

On the other hand, if you'll be using a Neo4j database defined elsewhere, you'll need to configure the Neo4j connection details in `application.properties` like this:

```
spring.neo4j.uri=bolt://some-neo4j-host:7687
spring.neo4j.authentication.username=neo4j
spring.neo4j.authentication.password=l3tM31n
```

Nothing else needs to change. When you run the application and start interacting with the Board Game Buddy application, chat history will be written as a set of entities and relationships to the Neo4j database. If you were to use the Neo4j Browser (<https://mng.bz/Nwov>) to explore the database, you might see a graph similar to figure 5.2.

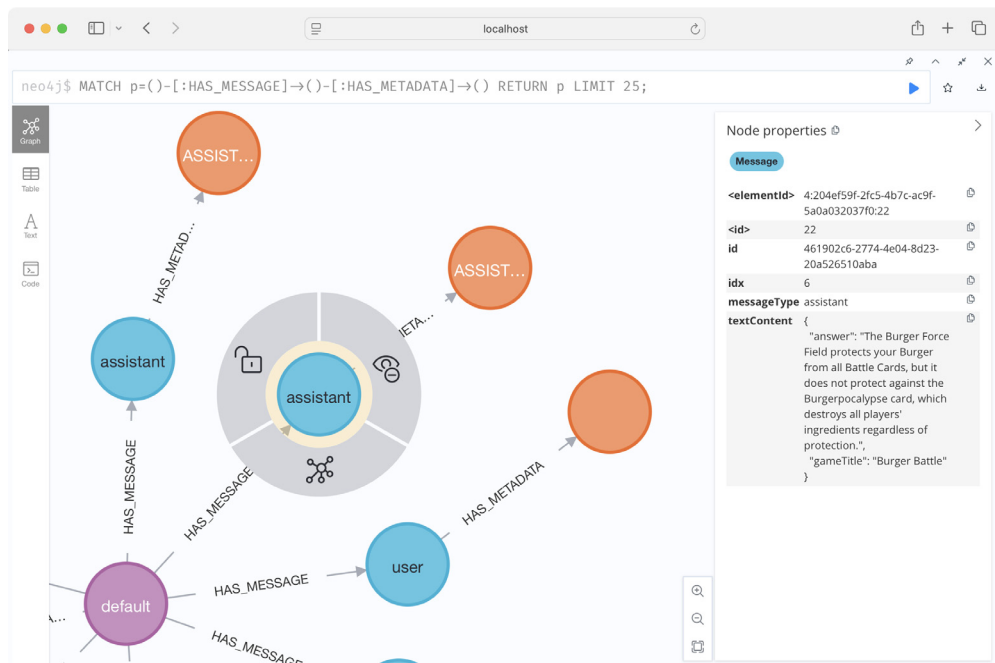


Figure 5.2 Viewing chat history in the Neo4j Browser

Figure 5.2 is zoomed in on the top-right quadrant of the visualization such that the default session (aka the default conversation) is in the bottom-left corner. Branching out from that entity node, you can see several user and assistant messages. One of the assistant messages is selected, and its properties are displayed to the right, including the text content that shows the answer to a question about the Burger Force Field card's limitations. Each of the user and assistant messages themselves is related to a metadata node, which carries additional meta-properties about each message.

The session/conversation entity is the core of a chat history. If you zoom out to see more of the graph, as in figure 5.3, you can see that in this database, three conversations are in play.

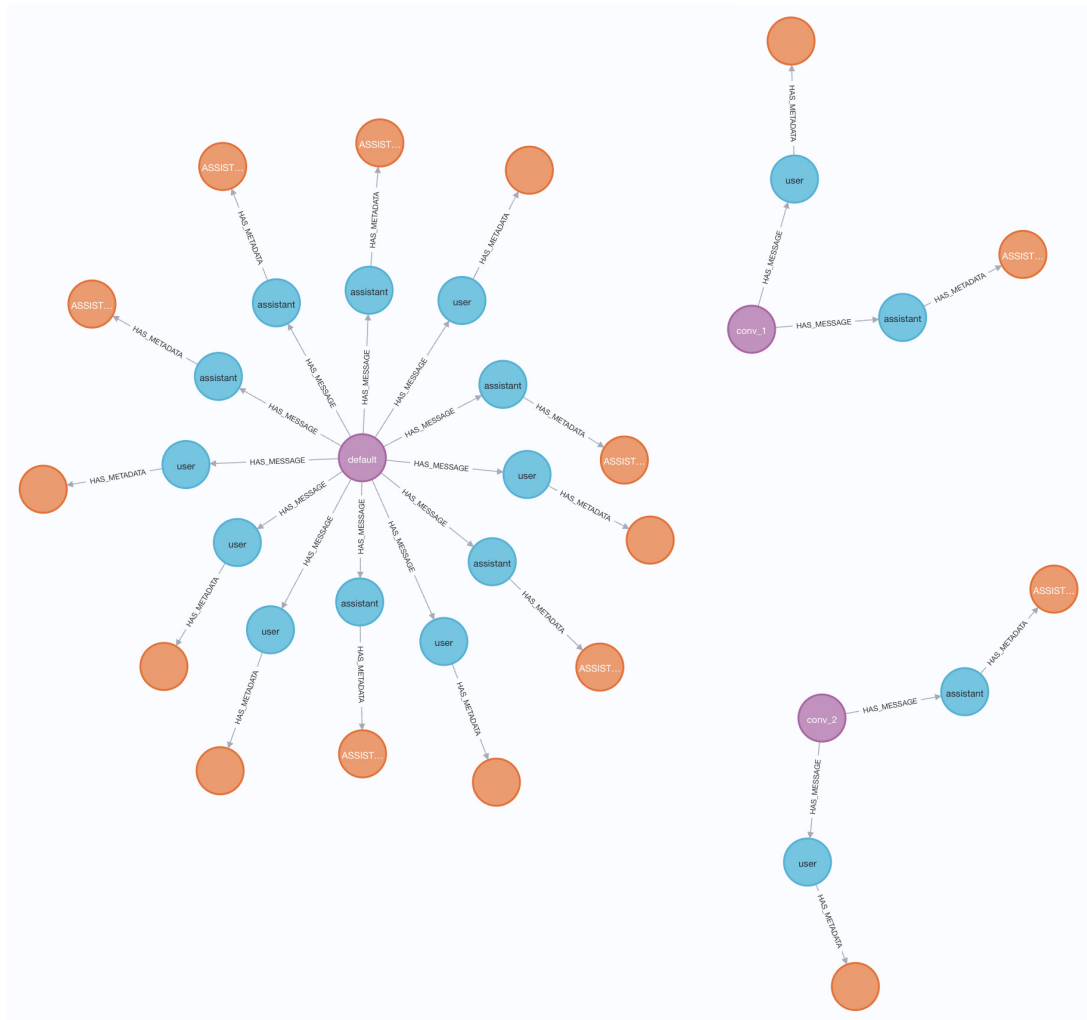


Figure 5.3 Multiple conversations visualized in Neo4j

Storing conversation history as a graph in Neo4j is an intriguing option, to be sure. But traditional relational databases have long been the workhorses of application persistence. Let's see how to use Spring AI's JDBC-backed chat memory repository to store chat history in a relational database.

JDBC

Spring AI's JDBC-based implementation of `ChatMemoryRepository` enables persistence of conversations to a relational database, including any of these databases:

- HSQLDB
- MySQL
- PostgreSQL
- SQL Server

It's up to you which of these databases you want to use for chat history; for the most part, there's not much difference in how you configure them. To illustrate how `JdbcChatMemoryRepository` works, let's pick PostgreSQL.

First, you'll need to use the PostgreSQL chat memory repository starter in your build instead of the Cassandra or Neo4j starters:

```
services:
  postgres:
    image: 'postgres:latest'
    environment:
      - 'POSTGRES_DB=mydatabase'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '5432:5432'
```

You'll also need the PostgreSQL JDBC driver:

```
runtimeOnly 'org.postgresql:postgresql'
```

Next, you'll need to explicitly configure a `ChatMemoryRepository` bean. Unlike Cassandra and Neo4j chat memory autoconfiguration, JDBC doesn't autoconfigure a `ChatMemoryRepository` bean. It needs to be created with details specific to the kind of database you are using, including a dialect that provides the SQL that works with the specific database. The following `chatMemoryRepository()` bean method shows how to define the bean for a PostgreSQL database:

```
@Bean
ChatMemoryRepository chatMemoryRepository(DataSource dataSource) {
    return JdbcChatMemoryRepository.builder()
        .dialect(new PostgresChatMemoryRepositoryDialect())
        .dataSource(dataSource)
        .build();
}
```

Here, a new instance of `PostgresChatMemoryRepositoryDialect` is given to the `dialect()` method of the builder. The builder is also given a `DataSource` to issue queries to the database.

By default, the schema will not be initialized with a table to hold the chat history. So, unless you are working with a pre-initialized database, you'll want to enable the schema initialization like this:

```
spring.ai.chat.memory.repository.jdbc.initialize-schema=always
```

Here, the `spring.ai.chat.memory.repository.jdbc.initialize-schema` property is set to `always` to indicate that the schema should always be initialized. If you're using an embedded database such as `HSQLDB` for tests or demo purposes, you may also choose to set the property to `embedded` to indicate that the schema should only be initialized when using the embedded database.

Aside from those small changes, everything else is the same as with `Cassandra` and `Neo4j`. Regardless of whether you chose to use `Cassandra`, `Neo4j`, or `JDBC`, you're now ready to try it out.

TRYING IT OUT

Fire up the application and try it out, just as you did when using the in-memory chat memory earlier:

```
$ http :8080/ask gameTitle="Burger Battle" \
      question="What is the Burger Force Field card?" \
      X_AI_CONVERSATION_ID:conversation_1 -b
{
  "answer": "The Burger Force Field card is a Battle Card in the Burger
            card game that allows a player's Burger to be protected from
            all other Battle Cards.",
  "gameTitle": "Burger Battle"
}
$ http :8080/ask gameTitle="Burger Battle" \
      question="Does it protect against Burgerpocalypse?" \
      X_AI_CONVERSATION_ID:conversation_1 -b
{
  "answer": "The Burger Force Field card does not protect against
            Burgerpocalypse as it specifically states that all players'
            ingredients are destroyed, regardless of protection.",
  "gameTitle": "Burger Battle"
}
```

As you can see, it worked! Regardless of which database you choose to back chat memory, the results are the same. Conversation history is stored persistently and used to give context as the conversation progresses. As long as the database isn't destroyed (say, by destroying the volume used by the Docker container), the application can stop and start without losing the conversation.

Chat memory can also be stored in a vector store such as `Qdrant`. In fact, Spring AI comes with a chat memory advisor implementation that does exactly that without

needing to write a custom `ChatMemoryRepository`. To wrap up this chapter, let's see how it works and what makes it different from the other chat memory advisors.

5.4.2 Storing chat memory in a vector store

Spring AI's `VectorStoreChatMemoryAdvisor` is very similar to `PromptChatMemoryAdvisor` in how it supplies chat memory as context to the LLM in a system prompt. But with regard to how it persists chat memory, it's quite different than either of the other advisors.

The most obvious difference is that it saves the chat memory to a vector store. But what might not be obvious is that because it writes to a vector store, embeddings are calculated for the chat memory so that they can be queried upon with a similarity search. Put simply, `VectorStoreChatMemoryAdvisor` applies the RAG pattern to chat memory. But instead of storing document chunks like the RAG you saw in chapter 4, `VectorStoreChatMemoryAdvisor` stores messages from the chat history.

Consequently, the chat memory fed to the LLM in the system prompt is more focused and relevant than if it were just the most recent so many messages. Rather than simply return the most recent several prompt messages, `VectorStoreChatMemoryAdvisor` only returns the messages in chat memory that are similar to the question being asked.

Despite its more advanced approach to chat memory, `VectorStoreChatMemoryAdvisor` is just as easy to use as either of the other two chat memory advisors.

Listing 5.4 Storing long-term memory in a vector store

```
@Bean
ChatClient chatClient(ChatClient.Builder chatClientBuilder,
    VectorStore vectorStore) {
    return chatClientBuilder
        .defaultAdvisors(
            VectorStoreChatMemoryAdvisor.builder(vectorStore).build(),
            QuestionAnswerAdvisor.builder(vectorStore)
                .searchRequest(SearchRequest.builder().build()).build())
        .build();
}
```

Adds
VectorStoreChatMemoryAdvisor ←

As was the case with `MessageChatMemoryAdvisor` and `PromptChatMemoryAdvisor`, `VectorStoreChatMemoryAdvisor` can be specified to the `ChatClient` in a single line of code. But instead of creating the advisor with an instance of some `ChatMemory` implementation, `MessageChatMemoryAdvisor` is created with a reference to the `VectorStore`, which you already have handy in the `ChatClient` bean's definition because you also needed it when you created `QuestionAnswerAdvisor` in chapter 4.

That's the only change you'll need to make to use `VectorStoreChatMemoryAdvisor`. But you can still set the conversation ID with the `CONVERSATION_ID` key to specify the conversation ID and chat memory size.

Summary

- LLMs have zero short-term memory and are unable on their own to carry on multiturn conversations.
- Applications can help the LLMs “remember” a conversation by taking note of every line of dialogue between a user and the assistant LLM and reminding the LLM of the chat history on each prompt.
- Spring AI provides three chat memory advisors, each with different strategies for supporting management of conversational memory.
- Long-term memory that outlasts a session can be achieved with `VectorStore-ChatMemoryAdvisor` or by creating a custom implementation of `ChatMemory`.
- When coupled with `QuestionAnswerAdvisor`, Spring AI’s chat memory advisors enable conversations with documents.